

Classi di Memorizzazione

Tutte le variabili e le funzioni del C hanno 2 attributi:

- il Tipo
- la **Classe di Memorizzazione**

Le 4 Classi di Memorizzazione possibili sono: `auto`, `extern`, `register`, `static`, `static extern`

Classe di Memorizzazione: `auto`

È la **classe di memorizzazione di default** e quindi può essere omessa

- All'ingresso in un blocco il sistema alloca memoria per le variabili automatiche;
 - pertanto, tali variabili sono considerate locali al blocco:
 - All'uscita dal blocco, il sistema libera la memoria assegnata alle variabili automatiche, causando la perdita dei loro valori
 - Al rientro nel blocco, il sistema alloca nuovamente la memoria senza però recuperare i valori precedenti

Classe di Memorizzazione: `extern`

Variabili (o funzioni) definite e allocate in altri file (esempio: un altro programma)

- Il compilatore assume che questa variabile esista e non alloca spazio per essa
- Il linker può generare un errore

Classe di Memorizzazione: `register`

Classe di memorizzazione che **ha come obiettivo l'aumento della velocità di esecuzione**:

- segnala al compilatore che la variabile corrispondente dovrebbe essere memorizzata in registri di memoria ad alta velocità
 - qualora il compilatore non possa allocare un registro fisico, viene utilizzata la classe **automatica** (NOTA: il compilatore dispone solo di una parte dei registri, che possono invece essere utilizzati dal sistema)
- Quando la velocità è importante, il programmatore può scegliere poche variabili alle quali viene fatto più frequentemente accesso (esempio: variabili di ciclo e parametri delle funzioni).
- La variabile `register` è di norma dichiarata nel punto più vicino possibile al punto in cui viene utilizzata, per consentire la massima disponibilità di registri fisici, utilizzati solo quando necessario

```
register int i;  
for(i = 0; i < LIMIT; ++i)  
...
```

Classe di Memorizzazione: static

Classe di memorizzazione che **permette a una variabile locale di mantenere il valore precedente al rientro in un blocco.**

```
void f(void) {  
    static int count = 0;  
    ++count;  
    ...  
}
```

- alla prima chiamata della funzione `count` viene inizializzata a zero;
- alle chiamate successive non viene più inizializzata, ma mantiene il valore che aveva alla precedente chiamata di funzione

Classe di Memorizzazione: static extern

Classe di memorizzazione che **fornisce meccanismo di 'privatezza'**: insieme di restrizioni sulla visibilità di variabili o funzioni (che sarebbero altrimenti accessibili), fondamentale per la modularità dei programmi.

- le variabili `static extern` **sono variabili** `extern` **con visibilità ristretta**:
 - **sono accessibili**: dal resto del file in cui sono dichiarate
 - **non sono disponibili**:
 - alle funzioni precedentemente definite all'interno del file
 - alle funzioni definite all'interno di file differenti

OSSERVAZIONI IMPORTANTI: Allocazione delle Variabili

- Le variabili `static` e `extern`, non inizializzate esplicitamente, **vengono inizializzate a 0 dal sistema**
 - anche array, stringhe, puntatori, strutture subiscono lo stesso trattamento: per array e stringhe ogni elemento viene posto a 0
 - **Al contrario**, normalmente le variabili `auto` e `register` **non vengono inizializzate dal sistema**, e quindi contengono inizialmente valori «sporchi»
 - Normalmente, **le variabili sono salvate in Memoria (BSS, Stack, Heap)** o in registri.
 - **BSS** (*Block Standard by Symbol*)
 - **Contiene Variabili Globali** e variabili con classe di memorizzazione `static`
 - Contenitore a disposizione di tutto il programma:
 - Istanziato all'inizio del programma
 - Deallocato alla fine del programma
 - **Stack**
 - **Contiene Variabili** con classe di memorizzazione `auto`
 - **N.B:** La dimensione dello Stack varia durante l'esecuzione del programma.
 - **Heap**
 - **N.B:** La dimensione dello Heap varia durante l'esecuzione del programma.
-